

QUESTION 1

Type: mcq

Prompt:

What is the primary benefit of using React Server Components (RSC)?

Guidance:

Focus on performance and bundle size implications.

Rubric:

Answers should mention reduced client-side JavaScript and direct backend access.

Solution:

RSCs run on the server, reducing the amount of JavaScript sent to the client and enabling direct

Allow Multiple: false

Options:

- ☒ Reduced client-side bundle size
- ☐ Faster client-side state updates
- ☐ Better support for CSS animations
- ☐ Automatic global state management

END QUESTION

QUESTION 2

Type: debug

Prompt:

A React component is re-rendering infinitely. Identify the cause and fix it.

Guidance:

Look at the useEffect dependency array and state updates.

Rubric:

Candidate should identify the circular dependency between state update and effect.

Solution:

The useEffect updates a state variable that is also in its dependency array (or causes a re-render)

Language: typescript

Starter Code:

```
````tsx
function UserProfile({ userId }) {
 const [user, setUser] = useState(null);

 useEffect(() => {
 fetchUser(userId).then(u => setUser(u));
 }); // <--- Missing dependency array!

 if (!user) return <div>Loading...</div>;
 return <div>{user.name}</div>;
}
...

```

Expected Outcome:

Candidate adds the missing dependency array `[userId]` to the useEffect hook.

END QUESTION

### QUESTION 3

Type: scenario

Prompt:

Architect a frontend for a real-time collaborative whiteboard.

Guidance:

Consider state management, conflict resolution, and performance.

Rubric:

Look for mentions of WebSockets, CRDTs (or operational transform), and canvas optimization.

Solution:

Use WebSockets for real-time sync, a CRDT library like Yjs for state consistency, and HTML5 Ca

Deliverable: Architecture diagram description

Constraints:

- Must support 50+ concurrent users
- Offline support required
- Low latency (< 100ms)

END QUESTION